

# Venus EBrake Emergency Router Audit Report

## Please Note

1. The analysis of the Severity is purely based on the smart contracts mentioned in the Audit Scope and does not include any other potential contracts deployed by the Owner. No applications or operations were reviewed for Severity. No product code has been reviewed.
2. Due to the time limit, the audit team did not do much in-depth research on the business logic of the project. It is more about discovering issues in the smart contracts themselves.

**Audit Period:** 2026/04/01 - 2026/04/07 (YYYY/MM/DD)

**Overall Risk:** **Low**

|                     |                                                                                                                                                                                                                                                                                                                                                                                                               |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Project Name</b> | Venus EBrake Emergency Router Audit                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Github</b>       | <a href="https://github.com/VenusProtocol/venus-periphery/pull/59">https://github.com/VenusProtocol/venus-periphery/pull/59</a><br><a href="https://github.com/VenusProtocol/venus-periphery/pull/60">https://github.com/VenusProtocol/venus-periphery/pull/60</a><br>Fix:<br><a href="https://github.com/VenusProtocol/venus-periphery/pull/62">https://github.com/VenusProtocol/venus-periphery/pull/62</a> |
| <b>Background</b>   | We have a small audit request here:<br><a href="https://github.com/VenusProtocol/venus-periphery/pull/59">https://github.com/VenusProtocol/venus-periphery/pull/59</a> <ul style="list-style-type: none"><li>• bg and info is in the PR</li><li>• Timeline: within this week (?) since its a small scope</li></ul>                                                                                            |

## Smart Contracts List:

| No. | File Name                | Verdict       | Details                                                                                         |
|-----|--------------------------|---------------|-------------------------------------------------------------------------------------------------|
| 1   | EBrake.sol               | Low           | [L01]-Ack<br>[I02]-Fixed<br>[I03]-Fixed<br>[I04]-Fixed<br>[I06]-Ack<br>[I07]-Ack<br>[I08]-Fixed |
| 2   | IEBrake.sol              | Green         |                                                                                                 |
| 3   | IComptroller.sol         | Green         |                                                                                                 |
| 4   | ICorePoolComptroller.sol | Green         |                                                                                                 |
| 5   | IILComptroller.sol       | Green         |                                                                                                 |
| 6   | DeviationSentinel.sol    | Informational | [I01]-Fixed<br>[I05]-Fixed                                                                      |

## Findings

### [L01] Premature resetMarketState Destroys Snapshot Permanently

**Contract** EBrake.sol

**Severity Level** Low

**Description**

**Description**

If governance calls `resetMarketState()` before restoring CF on the comptroller, the snapshot is cleared. If another emergency then occurs, `_snapshotCF` sees `currentCF == 0` (never restored) and skips snapshotting due to the first-write-wins guard. The original pre-incident CF value is permanently lost from on-chain records.

**Vulnerable Code**

JavaScript

```
// EBrake.sol:258-263 - _snapshotCF first-write-wins guard
function _snapshotCF(MarketState storage state, uint96 poolId, uint256
currentCF, uint256 currentLT) internal {
    if (state.poolCFs[poolId] == 0 && currentCF > 0) { // after
premature reset: poolCFs==0, currentCF==0 → skips
        state.poolCFs[poolId] = currentCF;
        state.poolLTs[poolId] = currentLT;
    }
}

// EBrake.sol:214-236 - resetMarketState clears everything
unconditionally
function resetMarketState(address market) external {
    _checkAccessAllowed("resetMarketState(address)");
    MarketState storage state = marketStates[market];
    state.borrowCap = 0;
    state.supplyCap = 0;
    state.borrowCapSnapshotted = false;
    state.supplyCapSnapshotted = false;
    // ... clears poolCFs and poolLTs
}
```

**Impact:**

Governance loses the pre-incident parameter values needed for recovery. Manual off-chain lookup through historical events is required, slowing incident response.

**Recommendation** Add a pre-reset guard to `resetMarketState` that covers both IL and Diamond paths. Only revert when a CF snapshot exists and the comptroller CF is still zero — this ensures the check fires exclusively when EBrake's snapshot has not been superseded by a governance restoration.

To avoid iterating over all pools unnecessarily, only inspect entries where poolCFs[i] != 0 (i.e., where EBrake actually stored a snapshot). Add a new custom error to the interface:

```
JavaScript

/// @notice Thrown when resetMarketState is called before CF has been
restored on the comptroller.
error CFNotYetRestored(address market, uint96 poolId);

function resetMarketState(address market) external {
    _checkAccessAllowed("resetMarketState(address)");
    MarketState storage state = marketStates[market];

    if (IS_ISOLATED_POOL) {
        if (state.poolCFs[0] != 0) {
            uint256 currentCF = IILComptroller(address(COMPTROLLER))
                .markets(market).collateralFactorMantissa;
            if (currentCF == 0) revert CFNotYetRestored(market, 0);
        }
    } else {
        uint96 corePoolId = COMPTROLLER.corePoolId();
        uint96 lastPoolId = COMPTROLLER.lastPoolId();
        for (uint96 i = corePoolId; i <= lastPoolId; ++i) {
            if (state.poolCFs[i] != 0) {
                (, uint256 currentCF, , , , ) =
                COMPTROLLER.poolMarkets(i, market);
                if (currentCF == 0) revert CFNotYetRestored(market,
                i);
            }
        }
    }

    // ... existing reset logic unchanged
}
```

|               |                                                                                                                                                                                                                                                                                                                                                                                         |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Status</b> | Acknowledged. resetMarketState was split into three independent functions — resetCFSnapshot, resetBorrowCapSnapshot, and resetSupplyCapSnapshot — decoupling CF and cap resets. However, the protective revert recommended by the report ("prevent reset before CF is restored") was not implemented. Premature calls to resetCFSnapshot can still permanently destroy the CF snapshot. |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

#### [I01] Missing CFZeroed event in handleDeviation function

|                 |                       |
|-----------------|-----------------------|
| <b>Contract</b> | DeviationSentinel.sol |
|-----------------|-----------------------|

|                       |               |
|-----------------------|---------------|
| <b>Severity Level</b> | Informational |
|-----------------------|---------------|

---

**Description****Description**

In the handleDeviation function, the function fires two operations when sentinel price is less than oracle price but only emits one event. Which is inconsistent with other conditions.

**Vulnerable Code**

JavaScript

```
function handleDeviation(IVToken market) external onlyKeeper {
    address underlyingToken = market.underlying();
    DeviationConfig memory config = tokenConfigs[underlyingToken];

    if (config.deviation == 0) revert MarketNotConfigured();
    if (!config.enabled) revert TokenMonitoringDisabled();

    (bool hasDeviation, uint256 oraclePrice, uint256
    sentinelPrice, ) = checkPriceDeviation(market);

    if (!hasDeviation) return;

    if (sentinelPrice > oraclePrice) {
        EBRAKE.pauseBorrow(address(market));
        emit BorrowPaused(address(market));
    } else {
        EBRAKE.setCFZero(address(market));
        EBRAKE.pauseSupply(address(market));
        //@@audit should also emit CFZero(market) event
        emit SupplyPaused(address(market));
    }
}
```

**Impact:**

Offchain service may fail to know if a market is CFZeroed.

---

**Recommendation** Add CFZero(market) event.**Status**

Fixed in [ed04846](#) and [df8bc15](#). DeviationSentinel now calls EBRAKE.decreaseCF(address(market), 0). EBrake emits a CollateralFactorDecreased event for each modified pool, providing complete on-chain records of CF actions.

JavaScript

```
DeviationAction action;
if (sentinelPrice > oraclePrice) {
    EBRAKE.pauseBorrow(address(market));
    action = DeviationAction.BorrowPaused;
} else {
```

```
EBRAKE.decreaseCF(address(market), 0);
EBRAKE.pauseSupply(address(market));
action = DeviationAction.SupplyPausedAndCFZeroed;
}
emit DeviationHandled(address(market), oraclePrice,
sentinelPrice, action);
```

## [I02] pauseFlashLoan emits event even when flash loans are already paused

**Contract** EBrake.sol

**Severity Level** Informational

**Description**

**Description**

The Ebrake.pauseFlashLoan() function emits the FlashLoanPaused event regardless of whether flash loans are already in a paused state. While the underlying call to the Comptroller's setFlashloanPaused() function will no-op if flash loans are already paused, the EBrake contract still emits the event.

### Vulnerable Code

```
JavaScript
function setFlashLoanPaused(bool paused) external {
    ensureAllowed("setFlashLoanPaused(bool)");

    // Check if value is actually changing
    if (flashLoanPaused == paused) {
        return; // No change needed
    }

    emit FlashLoanPauseChanged(flashLoanPaused, paused);
    flashLoanPaused = paused;
}
```

### Impact:

This is a purely code format issue and has no impact on contract state or functionality. However, it can produce misleading event logs that suggest a state transition occurred when none actually did, in the event Venus admins mistakenly pauses the flash loan twice.

**Recommendation** Consider returning early if flashLoanPaused == paused on the Comptroller side.

---

**Status**

Fixed in [6acf4fa](#). Added if (COMPTROLLER.flashLoanPaused()) return; as an early exit. When flash loans are already paused, both the Comptroller call and the event emission are skipped.

JavaScript

```
function pauseFlashLoan() external {
    _checkAccessAllowed("pauseFlashLoan()");

    // Idempotent: skip the external call AND the event when
    // already paused.
    // Keeps FlashLoanPaused a true state-change signal for
    // off-chain consumers
    // and matches the "EBrake duplicate calls are no-ops"
    // contract callers rely on.
    if (COMPTROLLER.flashLoanPaused()) return;

    COMPTROLLER.setFlashLoanPaused(true);
    emit FlashLoanPaused(msg.sender);
}
```

---

**[I03] Possible additional features for Ebrake mechanism of Core Pools**

---

**Contract** EBrake.sol

---

**Severity Level** Informational

---

**Description** **Description**

The EBrake contract currently provides only global emergency pause actions for borrowing and flash loans. Two possible new additional features can improve the precision of emergency pauses for borrowing and flashloans.

1. pauseBorrow(market) pauses the BORROW action globally via \_actionPaused[market][BORROW], which blocks borrowing across all pools simultaneously. However, the Core pool Comptroller enforces a second, independent core pool market check via poolBorrowAllowed(). In PolicyFacet, it checks that \_poolMarkets[index].isBorrowAllowed on a per-pool, per-market basis. EBrake has no wrapper for setIsBorrowAllowed(poolId, market, false). In a scenario where an e-mode pool (poolId > 0) experiences a emergency scenario, the only method for the Ebrake contract is to pause borrowing for that market in every pool. There is no way to isolate and disable borrowing in the affected e-mode core pool alone while leaving the core pool and other pools operational.
-

---

JavaScript

```
function borrowAllowed(address vToken, address borrower, uint256
borrowAmount) external returns (uint256) {
    // Pausing is a very serious situation - we revert to sound
    the alarms
    checkProtocolPauseState();
    checkActionPauseState(vToken, Action.BORROW);
    ensureListed(getCorePoolMarket(vToken));
    poolBorrowAllowed(borrower, vToken);
    //...
}
```

2. pauseFlashLoan() is a global kill switch that disables all flash loan execution. The Comptroller's FlashLoanFacet enforces a per-account whitelist via authorizedFlashLoan[msg.sender] checked within the FlashLoanFacet.executeFlashLoan function. If a single whitelisted flash loan executor is compromised, the only EBrake response is to pause flash loans entirely, given there is no method to invoke setWhiteListFlashLoanAccount(account, false) to revoke access for the specific whitelisted address.

JavaScript

```
function executeFlashLoan(
    address payable onBehalf,
    address payable receiver,
    VToken[] memory vTokens,
    uint256[] memory underlyingAmounts,
    bytes memory param
) external nonReentrant {
    if (flashLoanPaused) {
        revert FlashLoanPausedSystemWide();
    }

    ensureNonzeroAddress(onBehalf);

    uint256 len = vTokens.length;
    Market storage market;

    // vTokens array must not be empty
    if (len == 0) {
        revert NoAssetsRequested();
    }
    // Add maximum array length check to prevent gas limit issues
    if (len > MAX_FLASHLOAN_ASSETS) {
        revert TooManyAssetsRequested(len, MAX_FLASHLOAN_ASSETS);
    }
}
```

---



```

// All arrays must have the same length and not be zero
if (len != underlyingAmounts.length) {
    revert InvalidFlashLoanParams();
}

for (uint256 i; i < len; ++i) {
    market = getCorePoolMarket(address(vTokens[i]));
    if (!market.isListed) {
        revert MarketNotListed(address(vTokens[i]));
    }
    if (!(vTokens[i]).isFlashLoanEnabled()) {
        revert FlashLoanNotEnabled();
    }
    if (underlyingAmounts[i] == 0) {
        revert InvalidAmount();
    }
}

ensureNonzeroAddress(receiver);

if (!authorizedFlashLoan[msg.sender]) {
    revert SenderNotAuthorizedForFlashLoan(msg.sender);
}

```

**Recommendation** Consider the addition of two new functions to EBrake:

1. `disablePoolBorrow(uint96 poolId, address market)` that invokes `ICorePoolComptroller.setIsBorrowAllowed(poolId, market, false)` with its own ACM permission check and a `PoolBorrowDisabled` event.
2. `revokeFlashLoanAccess(address account)` that invokes `ICorePoolComptroller.setWhiteListFlashLoanAccount(account, false)` with a zero-address check, its own ACM permission check, and a `FlashLoanAccessRevoked` event.

If implemented, both functions should hardcode the restrictive value (`false`) to maintain the EBrake invariant that compromised authorized addresses can only freeze and never loosen protocol parameters.

**Status**

Fixed in [32878c6](#). Both functions are implemented. The boolean argument is hardcoded to `false` in both. `revokeFlashLoanAccess` includes a zero-address check. The corresponding function signatures were added to `ICorePoolComptroller`.

[I04] Cap snapshots and events recorded when requested cap equals current cap (no tightening)

|                 |            |
|-----------------|------------|
| <b>Contract</b> | EBrake.sol |
|-----------------|------------|

|                       |                      |
|-----------------------|----------------------|
| <b>Severity Level</b> | <b>Informational</b> |
|-----------------------|----------------------|

|                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Description</b> | <p><b>Description</b></p> <p>In setMarketBorrowCaps and setMarketSupplyCaps, the contract only reverts when newCap &gt; currentCap (&gt;). When newCap == currentCap, the call is allowed. The loop still records borrowCapSnapshot / supplyCapSnapshot and stores the current cap as the “pre-incident” snapshot before calling the comptroller, then invokes setMarketBorrowCaps / setMarketSupplyCaps with unchanged values and emits BorrowCapsDecreased / SupplyCapsDecreased. On-chain, comptroller caps are unchanged, but EBrake snapshot flags and events imply that a tightening action occurred.</p> <p>This is inconsistent observability for indexers, incident playbooks, and governance tooling.</p> <p>Note: Using &gt;= for the revert check would block newCap == 0 when the cap is already 0 (a case the team may intentionally allow for operational reasons); any fix should preserve that invariant if required.</p> |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

#### **Vulnerable Code**

Borrow caps:

```
JavaScript
for (uint256 i; i < marketsLen; ++i) {
    uint256 currentCap = comptroller.borrowCaps(markets[i]);
    if (newBorrowCaps[i] > currentCap) {
        revert CapExceedsCurrent(markets[i], currentCap,
newBorrowCaps[i]);
    }
    MarketState storage state = marketStates[markets[i]];
    if (!state.borrowCapSnapshot) {
        state.borrowCap = currentCap;
        state.borrowCapSnapshot = true;
    }
}

comptroller.setMarketBorrowCaps(markets, newBorrowCaps);
emit BorrowCapsDecreased(msg.sender, markets, newBorrowCaps);
```

Supply caps:

---

JavaScript

```
for (uint256 i; i < marketsLen; ++i) {
    uint256 currentCap = comptroller.supplyCaps(markets[i]);
    if (newSupplyCaps[i] > currentCap) {
        revert CapExceedsCurrent(markets[i], currentCap,
newSupplyCaps[i]);
    }
    MarketState storage state = marketStates[markets[i]];
    if (!state.supplyCapSnapshotted) {
        state.supplyCap = currentCap;
        state.supplyCapSnapshotted = true;
    }
}

comptroller.setMarketSupplyCaps(markets, newSupplyCaps);
emit SupplyCapsDecreased(msg.sender, markets, newSupplyCaps);
```

(Source: contracts/EmergencyBrake/EBrake.sol.)

**Impact:**

Integrity of off-chain interpretation, not direct theft. Authorized callers (per ACM) can produce snapshot state and events that overstate an emergency tightening when caps did not move. Risk is wrong VIP parameters, noisy alerts, or incorrect post-mortem reads. No additional unauthorized access beyond existing ACM trust.

---

|                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Recommendation</b> | <ol style="list-style-type: none"><li>1. Only set borrowCapSnapshotted / supplyCapSnapshotted (and stored pre-values) when newCap &lt; currentCap (strict decrease), while keeping &gt; (not &gt;=) for the revert rule so newCap == 0 on an already-zero cap remains valid if that is a required scenario.</li><li>2. Alternatively, emit a distinct event (or omit the decrease event) when newCap == currentCap.</li><li>3. Document explicitly that equality is a no-op on the comptroller but may still touch EBrake snapshots/events under the current code.</li></ol> |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

---

|               |                                                                                                                                                                                                                                                                |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Status</b> | Fixed in <a href="#">b5ba90a</a> . An anyDecreased flag was introduced. Snapshots, Comptroller calls, and events are only triggered when the new cap is strictly less than the current cap. Equal values are correctly treated as no-ops and skipped entirely. |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

---

## [I05] DeviationSentinel Events Are Redundant with EBrake Events and Lack Price Context

---

|                 |                   |
|-----------------|-------------------|
| <b>Contract</b> | DeviationSentinel |
|-----------------|-------------------|

---

|                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Severity Level</b> | <b>Informational</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Description</b>    | <p><b>Description</b></p> <p>handleDeviation emits BorrowPaused and SupplyPaused after routing emergency actions to EBrake. Both events carry only the market address, which is already captured by EBrake's ActionPaused(caller, market, action) (and CollateralFactorZeroed) events emitted in the same call chain. The net result is duplicate log entries with no additional information. More importantly, the data most useful for post-incident forensics — oracle price, sentinel price, and the deviation magnitude — is not recorded anywhere on-chain, forcing analysts to reconstruct these values off-chain.</p> <pre> JavaScript // DeviationSentinel - handleDeviation if (sentinelPrice &gt; oraclePrice) {     EBRAKE.pauseBorrow(address(market));     emit BorrowPaused(address(market));    // duplicates     ActionPaused(caller, market, BORROW) from EBrake } else {     EBRAKE.setCFZero(address(market));    // EBrake already emits     CollateralFactorZeroed     EBRAKE.pauseSupply(address(market));     emit SupplyPaused(address(market));    // duplicates     ActionPaused(caller, market, MINT) from EBrake } </pre> <p><b>Impact:</b></p> <p>No security impact. Redundant events waste approximately 375–750 extra gas per handleDeviation call. The absence of price data in emitted events forces off-chain monitoring infrastructure to re-query oracle contracts in order to reconstruct incident context, increasing operational complexity for incident response.</p> |
| <b>Recommendation</b> | <p>Replace BorrowPaused / SupplyPaused with a single event that captures price context and distinguishes the two response paths:</p> <pre> JavaScript event DeviationHandled(     address indexed market,     uint256 oraclePrice,     uint256 sentinelPrice,     bool borrowPaused    // true = borrow paused only; false = CF     zeroed + supply paused );  // inside handleDeviation </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |

```
emit DeviationHandled(address(market), oraclePrice, sentinelPrice,  
sentinelPrice > oraclePrice);
```

**Status** Fixed in [ed04846](#) and [df8bc15](#). Replaced BorrowPaused and SupplyPaused with a single DeviationHandled(market, oraclePrice, sentinelPrice, DeviationAction) event. Price context and action type are now recorded on-chain.

#### [I06] Gas bound on Diamond setCFZero(address) full-pool loop

**Contract** EBrake.sol

**Severity Level** **Informational**

**Description** **Description**  
On the Diamond comptroller path (IS\_ISOLATED\_POOL == false), setCFZero(address) iterates from corePoolId through lastPoolId, reading poolMarkets, snapshotting, and calling setCollateralFactor for each listed pool. If the number of pools is large or each call is expensive, the transaction may exceed the block gas limit and revert entirely, so the emergency action cannot complete in a single transaction. This does not require a malicious actor; it is an implementation scalability / availability constraint. Mitigation relies on setCFZero(address, uint96) (per-pool) or operational playbooks.

#### Vulnerable Code

This is the else arm of setCFZero(address) when IS\_ISOLATED\_POOL is false (full function starts around line 124 in contracts/EmergencyBrake/EBrake.sol).

```
JavaScript  
} else {  
    uint96 corePoolId = COMPTROLLER.corePoolId();  
    uint96 lastPoolId = COMPTROLLER.lastPoolId();  
    (bool isCorePoolListed, , , , , ) =  
    COMPTROLLER.poolMarkets(corePoolId, market);  
    if (!isCorePoolListed) revert MarketNotListed(corePoolId, market);  
    for (uint96 i = corePoolId; i <= lastPoolId; ++i) {  
        (bool isListed, uint256 currentCF, , uint256 currentLT, , , )  
        = COMPTROLLER.poolMarkets(i, market);  
        if (!isListed) continue;  
        _snapshotCF(state, i, currentCF, currentLT);  
        uint256 err = COMPTROLLER.setCollateralFactor(i, market, 0,  
currentLT);  
        if (err != 0) revert SetCollateralFactorFailed(err);
```

```
        emit CollateralFactorZeroed(msg.sender, market, i);
    }
}
```

**Impact:**

Availability / incident response:

Under sufficiently large lastPoolId range or gas cost per iteration, setCFZero(address) may never succeed as a single atomic call, delaying “zero CF across all relevant pools” until operators use per-pool calls or split work across transactions. Not a typical unauthorized fund-theft vector; callers remain subject to ACM permissions.

**Recommendation**

1. Add a bounded batch endpoint  
Introduce a function such as setCFZeroForPoolRange(address market, uint96 fromPoolId, uint96 toPoolId) (or equivalent with an explicit maxPools per transaction) so each call only iterates a subset of pool IDs, keeping gas within predictable limits. Update IEBrake and EBrake, and grant a separate ACM permission for this function distinct from setCFZero(address).
2. Document the gas / pool-count assumption and standard fallback (setCFZero(address, uint96) per pool) in runbooks.
3. Optionally add a view helper to surface pool span or rough cost signals, or a bounded batch API (trade-offs: ACM surface and complexity).
4. Fork-test gas on the target chain for realistic lastPoolId - corePoolId before mainnet reliance on the monolithic loop.

**Status**

Acknowledged. The team assessed that the current pool count (15 on BSC mainnet) is well within block gas limits. The single-pool overload decreaseCF(address, uint96, uint256) serves as a fallback should the pool count grow significantly.

## Fix Review

**Review Date:** 20260409

**Fix PR:** <https://github.com/VenusProtocol/venus-periphery/pull/62>

**Fix Review Summary**

| ID | Issue | Fixed? | Fix method |
|----|-------|--------|------------|
|----|-------|--------|------------|

|            |                                                                                                           |              |                                                                                                                                                                                                                                                                                                                                                                           |
|------------|-----------------------------------------------------------------------------------------------------------|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>L01</b> | Premature<br>resetMarketState<br>Destroys<br>Snapshot<br>Permanently                                      | Acknowledged | resetMarketState was split into three independent functions — resetCFSnapshot, resetBorrowCapSnapshot, and resetSupplyCapSnapshot — decoupling CF and cap resets. However, the protective revert recommended by the report ("prevent reset before CF is restored") was not implemented. Premature calls to resetCFSnapshot can still permanently destroy the CF snapshot. |
| <b>I01</b> | Missing<br>CFZeroed<br>event in<br>handleDeviation<br>function                                            | Fixed        | DeviationSentinel now calls EBrake.decreaseCF(address(market), 0). EBrake emits a CollateralFactorDecreased event for each modified pool, providing complete on-chain records of CF actions.                                                                                                                                                                              |
| <b>I02</b> | pauseFlashLoan<br>emits event<br>even when<br>flash loans are<br>already paused                           | Fixed        | Added if (COMPTROLLER.flashLoanPaused()) return; as an early exit. When flash loans are already paused, both the Comptroller call and the event emission are skipped.                                                                                                                                                                                                     |
| <b>I03</b> | Possible<br>additional<br>features for<br>Ebrake<br>mechanism of<br>Core Pools                            | Fixed        | Both functions are implemented. The boolean argument is hardcoded to false in both. revokeFlashLoanAccess includes a zero-address check. The corresponding function signatures were added to ICorePoolComptroller.                                                                                                                                                        |
| <b>I04</b> | Cap snapshots<br>and events<br>recorded when<br>requested cap<br>equals current<br>cap (no<br>tightening) | Fixed        | An anyDecreased flag was introduced. Snapshots, Comptroller calls, and events are only triggered when the new cap is strictly less than the current cap. Equal values are correctly treated as no-ops and skipped entirely.                                                                                                                                               |
| <b>I05</b> | DeviationSentinel<br>Events Are<br>Redundant with<br>EBrake Events<br>and Lack Price<br>Context           | Fixed        | Replaced BorrowPaused and SupplyPaused with a single DeviationHandled(market, oraclePrice, sentinelPrice, DeviationAction) event. Price context and action type are now recorded on-chain.                                                                                                                                                                                |

|            |                                                                            |              |                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------------|----------------------------------------------------------------------------|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>I06</b> | Gas bound on Diamond setCFZero(address) full-pool loop                     | Acknowledged | The team assessed that the current pool count (15 on BSC mainnet) is well within block gas limits. The single-pool overload decreaseCF(address, uint96, uint256) serves as a fallback should the pool count grow significantly.                                                                                                                                                                                                                           |
| <b>I07</b> | disablePoolBorrow Does Not Validate Market Listing Before Disabling Borrow | Acknowledged | The decreaseCF(address, uint96, uint256) function invokes poolMarkets to retrieve currentCF and currentLT, which are required for its core logic. The associated listing check is incidental rather than intentional. As setBorrowAllowed already enforces validation and reverts for unlisted markets, introducing an additional poolMarkets read in this context would result in an unnecessary SLOAD without providing additional security guarantees. |
| <b>I08</b> | revokeFlashLoanAccess Emits Event Even When Account Is Not Whitelisted     | Fixed        | The revokeFlashLoanAccess function now includes a check to verify whether the account is on the flash loan whitelist. If the account is not whitelisted, there is no need to perform the revoke operation.                                                                                                                                                                                                                                                |

## New Findings

### [I07] disablePoolBorrow Does Not Validate Market Listing Before Disabling Borrow

**Contract** EBrake.sol

**Severity Level** Informational

#### Description

#### Description

disablePoolBorrow calls COMPTROLLER.setBorrowAllowed directly without first calling poolMarkets(poolId, market) to verify that market is listed in poolId. When an invalid pool/market combination is passed, the call may silently succeed and write a phantom storage entry in the Comptroller (if the Comptroller performs no validation internally), emitting a PoolBorrowDisabled event for a combination that does not exist. If the Comptroller does revert internally, the resulting error message originates deep within the Comptroller and gives no clear indication that the parameters were invalid.

The sibling function decreaseCF(address, uint96, uint256) already reads poolMarkets and checks isListed before acting, making the two functions inconsistent.



---

## Vulnerable Code

JavaScript

```
// EBrake - disablePoolBorrow
function disablePoolBorrow(uint96 poolId, address market) external {
  _checkAccessAllowed("disablePoolBorrow(uint96,address)");
  // No listing validation
  COMPTROLLER.setIsBorrowAllowed(poolId, market, false);
  emit PoolBorrowDisabled(msg.sender, poolId, market);
}

// EBrake - decreaseCF (contrast: validation present)
function decreaseCF(address market, uint96 poolId, uint256 newCF)
external {
  ...
  (bool isListed, uint256 currentCF, , uint256 currentLT, , , ) =
  COMPTROLLER.poolMarkets(poolId, market);
  if (!isListed) revert MarketNotListed(poolId, market);
  ...
}
```

### Impact:

During incident response, an operator who passes the wrong poolId will receive a successful transaction receipt and see a PoolBorrowDisabled event, while the target pool's borrowing remains completely unrestricted. This false confirmation can delay the actual remediation, prolonging exposure during an active incident.

---

|                       |                                                                                          |
|-----------------------|------------------------------------------------------------------------------------------|
| <b>Recommendation</b> | Read and validate isListed before calling the Comptroller, and add an idempotency guard: |
|-----------------------|------------------------------------------------------------------------------------------|

JavaScript

```
function disablePoolBorrow(uint96 poolId, address market) external {
  _checkAccessAllowed("disablePoolBorrow(uint96,address)");
  (bool isListed, , , , , bool isBorrowAllowed) =
  COMPTROLLER.poolMarkets(poolId, market);
  if (!isListed) revert MarketNotListed(poolId, market);
  if (!isBorrowAllowed) return;
  COMPTROLLER.setIsBorrowAllowed(poolId, market, false);
  emit PoolBorrowDisabled(msg.sender, poolId, market);
}
```

---

### Status

Acknowledged. The decreaseCF(address, uint96, uint256) function invokes poolMarkets to retrieve currentCF and currentLT, which are required for its core logic. The associated listing check is incidental rather than intentional. As setIsBorrowAllowed already enforces validation and reverts for unlisted markets,

---

---

introducing an additional poolMarkets read in this context would result in an unnecessary SLOAD without providing additional security guarantees.

---

#### [I08] revokeFlashLoanAccess Emits Event Even When Account Is Not Whitelisted

**Contract** EBrake.sol

**Severity Level** Informational

**Description**

revokeFlashLoanAccess does not check whether the target account is actually present in the flash loan whitelist before calling COMPTROLLER.setWhiteListFlashLoanAccount. As a result, FlashLoanAccessRevoked is emitted regardless of whether the account ever held flash loan access.

**Vulnerable Code**

JavaScript

```
// EBrake - revokeFlashLoanAccess
function revokeFlashLoanAccess(address account) external {
    _checkAccessAllowed("revokeFlashLoanAccess(address)");
    if (account == address(0)) revert ZeroAddress();
    // No check that account is actually whitelisted
    COMPTROLLER.setWhiteListFlashLoanAccount(account, false);
    emit FlashLoanAccessRevoked(msg.sender, account);
}
```

**Impact:**

Calling this function for an address that was never whitelisted produces a FlashLoanAccessRevoked log entry on-chain, misleading off-chain monitoring and post-incident auditors into believing the account previously held flash loan privileges. This introduces noise into incident timelines and forensic analysis. There is no risk of fund loss.

**Recommendation** Query the whitelist state via authorizedFlashLoan(account) before acting, and return early if the account is not whitelisted:

JavaScript

```
function revokeFlashLoanAccess(address account) external {
    _checkAccessAllowed("revokeFlashLoanAccess(address)");
    if (account == address(0)) revert ZeroAddress();
    if (!COMPTROLLER.authorizedFlashLoan(account)) return;
}
```

---

---

```
    COMPTROLLER.setWhiteListFlashLoanAccount(account, false);  
    emit FlashLoanAccessRevoked(msg.sender, account);  
}
```

---

**Status**

Fixed in 9af6c82. The revokeFlashLoanAccess function now includes a check to verify whether the account is on the flash loan whitelist. If the account is not whitelisted, there is no need to perform the revoke operation.

---